

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimedia

Traitement des Données Multimodales

Data Augmentation Multimodale

Guide Complet des Techniques d'Augmentation de Données Textuelles, Visuelles et Audio

Réalisé par :
Prof. Abdellah ADIB

August 16, 2026

Contents

1	Introduction	3
1.1	Contexte et Objectifs	3
1.2	La Data Augmentation Multimodale	3
1.3	Le Principe Invariant de la Data Augmentation	3
2	Augmentation Textuelle	5
2.1	Principe Fondamental	5
2.1.1	Architecture du Pipeline d'Augmentation Textuelle	5
2.2	Installation et Configuration	5
2.3	Word and Sentence Shuffling	6
2.4	Word Replacement	7
2.5	Random Insertion et Random Deletion	7
2.6	Bonnes Pratiques	8
3	Augmentation d'Images	10
3.1	Principe Fondamental	10
3.1.1	Architecture du Pipeline d'Augmentation d'Images	10
3.2	Installation	10
3.3	Transformations Géométriques	11
3.4	Transformation de l'Espace Colorimétrique	11
3.5	Filtres à Noyau	12
3.6	Random Erasing et Mixing Images	12
3.7	Bonnes Pratiques	13
4	Augmentation Audio	14
4.1	Principe Fondamental	14
4.1.1	Architecture du Pipeline d'Augmentation Audio	14
4.2	Installation	14
4.3	Time Stretching et Amplitude Scaling	15
4.4	Noise Addition	15
4.5	Reverberation	16
4.6	Time and Frequency Masking (SpecAugment)	17
4.7	Bonnes Pratiques	17
5	Conclusion	18
5.1	Tableau Comparatif des Techniques	18
5.2	Erreurs Fréquentes et Solutions	18
5.3	Bonnes Pratiques Générales	19
5.4	Checklist de Reproductibilité	19
5.5	Arbre de Décision	20
6	Bibliographie et Ressources	21

6.1	Ouvrages de Référence	21
6.2	Documentation Officielle	21
6.3	Tutoriels Recommandés	21
6.4	Jeux de Données pour Pratiquer	22
6.5	Aspects Légaux et Éthiques	22

Licence IRM

Chapitre 1 Introduction

1.1 Contexte et Objectifs

Les modèles d'apprentissage automatique — en particulier les réseaux profonds — nécessitent de grands volumes de données pour généraliser correctement. Or les jeux de données annotés restent souvent limités, coûteux à constituer ou déséquilibrés entre classes. La **Data Augmentation** répond à ce problème : elle désigne l'ensemble des techniques permettant de générer de nouvelles variantes plausibles d'une donnée existante — texte, image ou audio — sans en modifier le sens ni le label associé.

Ce rapport présente un guide pratique de la Data Augmentation multimodale, structuré en trois chapitres correspondant aux trois modalités principales : le texte, l'image et l'audio. Chaque chapitre expose les concepts fondamentaux, les bibliothèques Python de référence et des exemples de code complets.

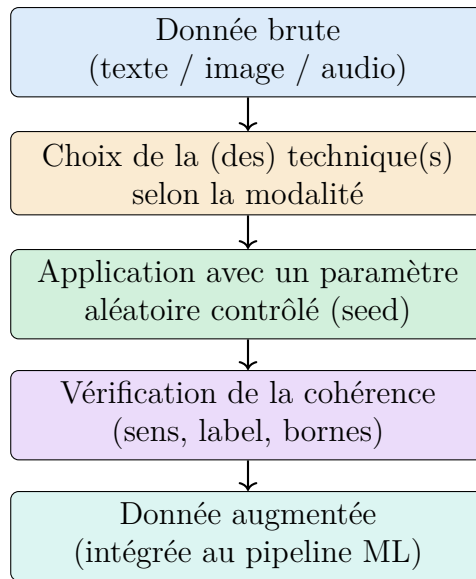
1.2 La Data Augmentation Multimodale

Le concept de Data Augmentation désigne tous les procédés permettant de perturber une donnée existante — structurellement ou statistiquement — afin d'enrichir un jeu d'entraînement sans en altérer la signification. Trois grandes familles de techniques couvrent l'ensemble des modalités abordées dans ce module.

1. **Augmentation Textuelle** : réordonnancement (shuffling), substitution lexicale (replacement), ajout (insertion) et suppression (deletion) de mots ou de phrases. Bibliothèques : NLTK, nlpaug.
2. **Augmentation d'Images** : transformations géométriques, transformation de l'espace colorimétrique, filtres à noyau, random erasing et mixing d'images. Bibliothèques : OpenCV, NumPy, Albumentations.
3. **Augmentation Audio** : étirement temporel (time stretching), ajout de bruit, mise à l'échelle de l'amplitude, réverbération et masquage temps/fréquence (SpecAugment). Bibliothèques : Librosa, Pydub, SoundFile.

1.3 Le Principe Invariant de la Data Augmentation

Quelle que soit la modalité, une transformation d'augmentation doit modifier la **représentation** de la donnée tout en préservant son **sens** et son **label** — une image de chat pivotée reste une image de chat, une phrase dont l'ordre des mots a été permuté conserve son intention.

**Conseil**

Choisissez la famille de techniques en fonction de la **nature** de la donnée :

- Donnée = texte, sous-titres, transcriptions → Augmentation Textuelle (Chapitre 2)
- Donnée = image, vidéo, frame → Augmentation d'Images (Chapitre 3)
- Donnée = son, parole, musique → Augmentation Audio (Chapitre 4)

Attention

L'augmentation ne s'applique **jamais** au jeu de **test** — uniquement au jeu d'entraînement, afin de ne pas biaiser l'évaluation du modèle.

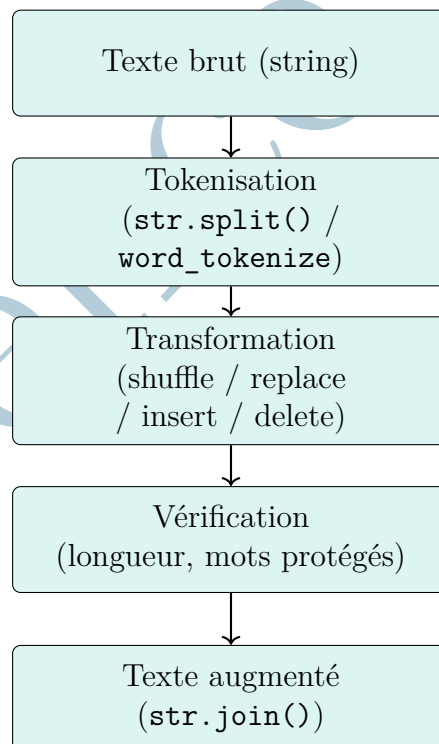
Chapitre 2 Augmentation Textuelle

2.1 Principe Fondamental

L'augmentation textuelle consiste à générer des variantes d'une phrase ou d'un document en perturbant légèrement sa **structure** (ordre) ou son **contenu** (vocabulaire), sans altérer l'information sémantique globale. Les techniques agissent au niveau du **mot** (word-level) ou de la **phrase** (sentence-level), selon le type de tâche (classification, reconnaissance d'entités, résumé).

NLTK (Natural Language Toolkit) fournit les briques de base (tokenisation, accès à WordNet pour les synonymes). **nlpaug** encapsule des techniques d'augmentation prêtes à l'emploi avec des back-ends embeddings/BERT pour des projets plus avancés.

2.1.1 Architecture du Pipeline d'Augmentation Textuelle



2.2 Installation et Configuration

Listing 2.1: Installation des bibliotheques de reference

```
pip install nltk nlpaug
```

Listing 2.2: Téléchargement des ressources NLTK nécessaires

```
import nltk

# Ressources indispensables (à exécuter une seule fois par
# machine)
nltk.download('wordnet')      # base lexicale pour les synonymes
nltk.download('punkt')       # tokeniseur de mots et de phrases
```

Attention

Sans les ressources NLTK téléchargées, tout appel à `wordnet.synsets` ou `word_tokenize` levera une erreur `LookupError`. Cette étape est obligatoire avant toute utilisation.

2.3 Word and Sentence Shuffling

Définition

Le **shuffling** consiste à permuter aléatoirement l'ordre des mots dans une phrase ou l'ordre des phrases dans un document, sans modifier le vocabulaire employé.

Utilité

Renforce la robustesse d'un modèle face à des ordres de mots atypiques et convient particulièrement aux approches de type *bag-of-words*, où l'ordre pèse peu sur la classification.

Listing 2.3: Word et Sentence Shuffling

```
import random

def shuffle_words(sentence: str, seed: int = 42) -> str:
    """Permute aléatoirement l'ordre des mots d'une phrase."""
    random.seed(seed)
    words = sentence.split()
    random.shuffle(words)
    return " ".join(words)

def shuffle_sentences(text: str) -> str:
    """Permute aléatoirement l'ordre des phrases d'un texte."""
    phrases = text.split(".")
    random.shuffle(phrases)
    return ". ".join(phrases)

texte = "Le chat dort. Le chien court. L'oiseau chante."
print(shuffle_sentences(texte))
# "L'oiseau chante. Le chat dort. Le chien court."
```

2.4 Word Replacement

Définition

Le **word replacement** substitue un mot par un synonyme, en s'appuyant sur une base lexicale (WordNet) ou sur des embeddings contextuels.

Utilité

Enrichit le vocabulaire perçu par le modèle sans changer le sens, et simule des reformulations naturelles rencontrées en production.

Listing 2.4: Word Replacement par synonymes WordNet

```
import nltk, random
from nltk.corpus import wordnet
nltk.download('wordnet')

def get_synonym(word: str) -> str:
    synsets = wordnet.synsets(word, lang='fra')
    if not synsets:
        return word
    lemmes = synsets[0].lemmas(lang='fra')
    return lemmes[0].name() if lemmes else word

def word_replacement(sentence: str, p: float = 0.3) -> str:
    """Remplace chaque mot par un synonyme avec une
    probabilité p."""
    words = sentence.split()
    return " ".join(get_synonym(w) if random.random() < p
                    else w for w in words)

print(word_replacement("Le chat mange rapidement"))
```

2.5 Random Insertion et Random Deletion

Définition

Le **random insertion** ajoute des mots aléatoires issus d'un vocabulaire donné ; le **random deletion** supprime des mots avec une probabilité p fixée.

Utilité

L'insertion simule du bruit rédactionnel (mots parasites) ; la suppression simule des textes incomplets ou bruités (fautes de frappe, coupures de flux audio-texte).

Fonction	Fonctionnalite	Instruction
<code>random.shuffle()</code>	Permuter une liste de mots	<code>random.shuffle(words)</code>
<code>random.choice()</code>	Tirer un mot du vocabulaire	<code>random.choice(VOCAB)</code>
<code>list.insert()</code>	Inserer a une position donnee	<code>words.insert(pos, mot)</code>
<code>random.random()</code>	Tirer une probabilite [0,1]	<code>if random.random() > p</code>

Listing 2.5: Random Insertion et Random Deletion

```

import random

VOCAB = ["souvent", "vraiment", "toujours", "parfois"]

def random_insertion(sentence: str, n: int = 1) -> str:
    """Insere n mots aleatoires issus du vocabulaire."""
    words = sentence.split()
    for _ in range(n):
        pos = random.randint(0, len(words))
        words.insert(pos, random.choice(VOCAB))
    return " ".join(words)

def random_deletion(sentence: str, p: float = 0.2) -> str:
    """Supprime chaque mot avec une probabilite p."""
    words = sentence.split()
    kept = [w for w in words if random.random() > p]
    return " ".join(kept) if kept else random.choice(words)

print(random_insertion("Le chat dort sur le canape", n=2))
print(random_deletion("Le traitement des donnees est
    complexe"))

```

2.6 Bonnes Pratiques

- Ne jamais modifier les mots-clés porteurs du label (négation, entités nommées).
- Limiter la probabilité p à 10–30 % pour préserver la lisibilité du texte.
- Fixer les graines aléatoires (`random.seed`) pour la reproductibilité.
- Sauvegarder le texte original avec la transformation appliquée en métadonnée.
- Combiner au maximum deux techniques par phrase pour éviter un texte incohérent.

Attention

La lemmatisation WordNet reste conçue pour l'anglais ; pour le français, préférez les synsets `lang='fra'` d'Open Multilingual WordNet ou un lemmatiseur dédié (spaCy + `fr_core_news_sm`).

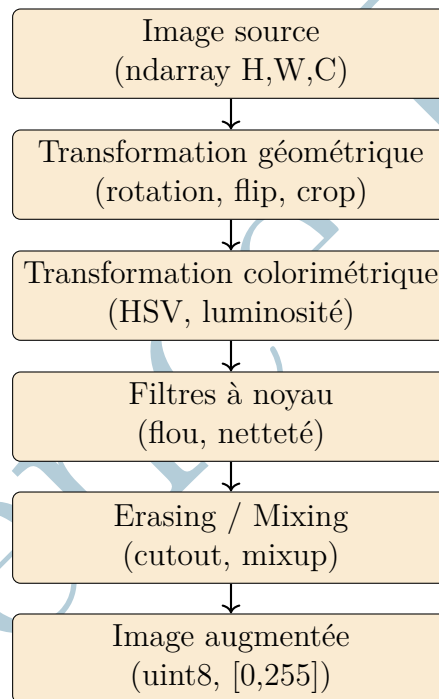
Licence IRM

Chapitre 3 Augmentation d'Images

3.1 Principe Fondamental

L'augmentation d'images consiste à appliquer des transformations spatiales, colorimétriques ou structurelles à une image afin de simuler la variabilité rencontrée en conditions réelles (angle, lumière, occlusion). **OpenCV** et **NumPy** sont les outils de référence : les transformations s'appliquent directement sur le tableau `ndarray` (H,W,C) de l'image.

3.1.1 Architecture du Pipeline d'Augmentation d'Images



3.2 Installation

Listing 3.1: Installation des bibliotheques vision

```
pip install opencv-python numpy
pip install albumentations # Pipelines d'augmentation
                             optimises (optionnel)
```

Listing 3.2: Importation et verification

```
import cv2
import numpy as np

print(cv2.__version__)
```

3.3 Transformations Géométriques

Définition

Les **transformations géométriques** modifient la position, l'orientation ou le cadrage des pixels : rotation, symétrie (flip), recadrage (crop), translation.

Utilité

Rendent le modèle invariant à l'angle de prise de vue, au cadrage et à l'orientation de l'objet photographié.

Listing 3.3: Transformations geometriques avec OpenCV

```
import cv2, numpy as np

img = cv2.imread("image.jpg")
h, w = img.shape[:2]

# Rotation aleatoire
angle = np.random.uniform(-25, 25)
M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))

# Flip horizontal
flipped = cv2.flip(img, 1)

cv2.imwrite("rotated.jpg", rotated)
```

3.4 Transformation de l'Espace Colorimétrique

Définition

La **transformation colorimétrique** perturbe la teinte, la saturation ou la luminosité d'une image via une conversion vers l'espace HSV.

Utilité

Simule des variations d'éclairage, de capteur ou de balance des blancs rencontrées en conditions réelles d'acquisition.

Listing 3.4: Transformation de l'espace colorimetrique HSV

```
import cv2, numpy as np

img = cv2.imread("image.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV).astype(np.float32)

# Saturation et luminosite aleatoires
```

```
hsv[..., 1] *= np.random.uniform(0.6, 1.4)
hsv[..., 2] *= np.random.uniform(0.7, 1.3)
hsv = np.clip(hsv, 0, 255).astype(np.uint8)

augmented = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
cv2.imwrite("color_aug.jpg", augmented)
```

3.5 Filtres à Noyau

Définition

Les **filtres à noyau** appliquent une convolution 2D sur l'image : flou (lissage), netteté (sharpen) ou détection de contours.

Utilité

Simulent un flou de mise au point ou renforcent les contours pour améliorer la robustesse du modèle aux conditions optiques dégradées.

Listing 3.5: Filtres à noyau : flou et netteté

```
import cv2, numpy as np

img = cv2.imread("image.jpg")

# Flou gaussien
blurred = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)

# Filtre de netteté (sharpen)
kernel = np.array([[0, -1, 0],
                   [-1, 5, -1],
                   [0, -1, 0]])
sharpened = cv2.filter2D(img, -1, kernel)

cv2.imwrite("blurred.jpg", blurred)
```

3.6 Random Erasing et Mixing Images

Définition

Le **random erasing** occulte une zone rectangulaire aléatoire de l'image ; le **mixing** (mixup / cutmix) fusionne linéairement deux images et leurs labels.

Utilité

Le random erasing renforce la robustesse aux occlusions partielles ; le mixing régularise le modèle en l'entraînant sur des combinaisons convexes d'exemples.

Listing 3.6: Random Erasing et Mixup

```
import cv2, numpy as np

def random_erasing(img, p=0.5, area_ratio=0.1):
    if np.random.rand() > p:
        return img
    h, w = img.shape[:2]
    eh, ew = int(h * area_ratio), int(w * area_ratio)
    y, x = np.random.randint(0, h - eh), np.random.randint(0,
        w - ew)
    img[y:y + eh, x:x + ew] = np.random.randint(0, 255, (eh,
        ew, 3))
    return img

def mixup(img1, img2, alpha=0.4):
    lam = np.random.beta(alpha, alpha)
    return (lam * img1 + (1 - lam) * img2).astype(np.uint8)
```

3.7 Bonnes Pratiques

- Adaptez les transformations au domaine (un flip vertical est déconseillé sur des panneaux routiers contenant du texte).
- Vérifiez que les *bounding boxes* suivent les transformations géométriques.
- Toujours `np.clip()` et forcer `astype(np.uint8)` après transformation.
- Appliquez l'augmentation en *online* pendant l'entraînement (par batch).
- Préférez `albumentations` pour des pipelines optimisés et composables.

Attention

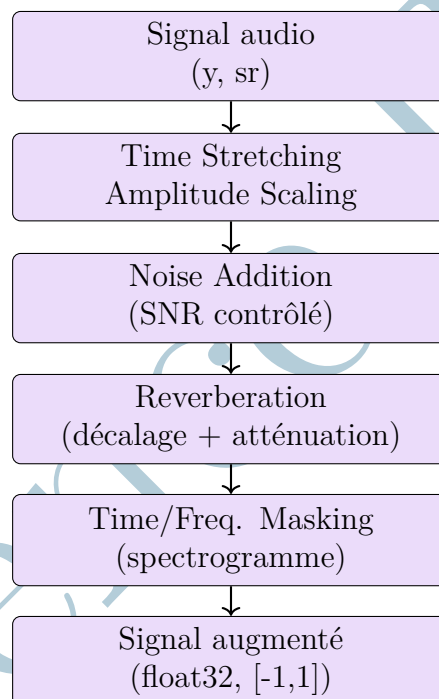
Une rotation qui laisse apparaître des bords noirs peut introduire un artefact appris par le modèle. Utilisez `borderMode=cv2.BORDER_REFLECT` pour l'éviter.

Chapitre 4 Augmentation Audio

4.1 Principe Fondamental

L'augmentation audio consiste à perturber le signal temporel ou sa représentation spectrale (spectrogramme) pour simuler des conditions d'enregistrement variées : bruit ambiant, distance, débit de parole. **Librosa** manipule le signal et le spectrogramme ; **Pydub** facilite le chargement et l'export multi-formats (WAV, MP3).

4.1.1 Architecture du Pipeline d'Augmentation Audio



4.2 Installation

Listing 4.1: Installation des bibliothèques audio

```
pip install librosa soundfile pydub
```

Listing 4.2: Importation et chargement d'un signal

```
import librosa
import soundfile as sf

y, sr = librosa.load("audio.wav", sr=None)
print(sr, y.shape)
```

4.3 Time Stretching et Amplitude Scaling

Définition

Le **time stretching** dilate ou compresse la durée d'un signal sans modifier sa hauteur tonale ; l'**amplitude scaling** multiplie le signal par un gain aléatoire (volume).

Utilité

Simule des débits de parole différents et des niveaux de volume variables rencontrés en conditions réelles d'enregistrement.

Listing 4.3: Time Stretching et Amplitude Scaling

```
import librosa, numpy as np, soundfile as sf

y, sr = librosa.load("audio.wav", sr=None)

# Etirement temporel (hauteur preserveree)
y_stretch = librosa.effects.time_stretch(y, rate=1.2)

# Mise a l'echelle de l'amplitude (volume)
gain = np.random.uniform(0.5, 1.5)
y_scaled = np.clip(y * gain, -1.0, 1.0)

sf.write("stretched.wav", y_stretch, sr)
```

4.4 Noise Addition

Définition

Le **noise addition** ajoute un bruit gaussien contrôlé par un rapport signal/bruit (SNR) cible exprimé en décibels.

Utilité

Rend le modèle robuste au bruit ambiant (rue, ventilateur, foule) rencontré hors studio.

Listing 4.4: Ajout de bruit controle par SNR

```
import librosa, numpy as np, soundfile as sf

y, sr = librosa.load("audio.wav", sr=None)

def add_noise(signal, snr_db=15):
    p_signal = np.mean(signal ** 2)
    p_noise = p_signal / (10 ** (snr_db / 10))
    noise = np.random.normal(0, np.sqrt(p_noise), signal.shape)
```



```

    return signal + noise

y_noisy = add_noise(y, snr_db=10)
sf.write("noisy.wav", y_noisy, sr)

```

4.5 Reverberation

Définition

La **réverbération** simule l'acoustique d'une pièce en ajoutant une version retardée et atténuée du signal original.

Utilité

Simule l'acoustique d'une salle, d'un couloir ou d'un espace extérieur, utile pour la reconnaissance vocale en conditions réelles.

Fonction	Fonctionnalité	Instruction
<code>time_stretch()</code>	Dilater/compresser la durée	<code>effects.time_stretch(y, rate)</code>
<code>np.random.normal()</code>	Générer un bruit gaussien	<code>np.random.normal(0, sigma)</code>
<code>np.clip()</code>	Borner l'amplitude	<code>np.clip(y, -1.0, 1.0)</code>
<code>melspectrogram()</code>	Calculer le spectrogramme mel	<code>feature.melspectrogram(y, sr)</code>

Listing 4.5: Simulation de reverberation

```

import librosa, numpy as np, soundfile as sf

y, sr = librosa.load("audio.wav", sr=None)

def add_reverb(signal, decay=0.4, delay_ms=60, sr=16000):
    delay = int(sr * delay_ms / 1000)
    echo = np.zeros_like(signal)
    echo[delay:] = signal[:-delay] * decay
    return signal + echo

y_reverb = add_reverb(y, decay=0.4, delay_ms=60, sr=sr)
sf.write("reverb.wav", y_reverb, sr)

```

4.6 Time and Frequency Masking (SpecAugment)

Définition

Le **Time/Frequency Masking** (SpecAugment) masque des bandes temp-s/fréquence directement sur le spectrogramme, en les mettant à zéro.

Utilité

Rend les modèles de reconnaissance vocale robustes aux pertes d'information locales, sans nécessiter de données supplémentaires.

Listing 4.6: Time and Frequency Masking sur spectrogramme mel

```
import librosa, numpy as np

y, sr = librosa.load("audio.wav", sr=None)
S = librosa.feature.melspectrogram(y=y, sr=sr)

def freq_mask(spec, F=15):
    f = np.random.randint(0, F)
    f0 = np.random.randint(0, spec.shape[0] - f)
    spec[f0:f0 + f, :] = 0
    return spec

def time_mask(spec, T=25):
    t = np.random.randint(0, T)
    t0 = np.random.randint(0, spec.shape[1] - t)
    spec[:, t0:t0 + t] = 0
    return spec

S_masked = time_mask(freq_mask(S.copy()))
```

4.7 Bonnes Pratiques

- Toujours clip le signal dans $[-1.0, 1.0]$ après transformation.
- Conserver le `sample rate` (`sr`) d'origine pour la cohérence du pipeline.
- Ne pas masquer plus de 20 % du spectrogramme (perte d'information excessive).
- Utiliser `audiomentations` pour des pipelines composables et rapides.
- Sauvegarder les paramètres appliqués (SNR, gain, delay) en métadonnées.

Attention

Le Time/Frequency Masking s'applique sur le **spectrogramme**, pas sur le signal brut — à ne pas confondre avec les autres techniques qui opèrent sur `y` directement.

Chapitre 5 Conclusion

5.1 Tableau Comparatif des Techniques

Modalité	Technique	Objectif	Outil Python	Sortie
Texte	Shuffling	Robustesse a l'ordre	<code>random.shuffle</code>	str augmentee
	Replacement	Enrichir vocabulaire	<code>wordnet.synset</code>	str augmentee
	Insertion	Simuler bruit texte	<code>list.insert</code>	str augmentee
	Deletion	Simuler texte incomplet	<code>random.random</code>	str augmentee
	Geometrique	Invariance angle/cadrage	<code>cv2.warpAffine</code>	ndarray (H,W,C)
	Colorimetrique	Invariance eclairage	<code>cv2.cvtColor</code>	ndarray uint8
	Filtres a noyau	Robustesse optique	<code>cv2.filter2D</code>	ndarray uint8
	Random Erasing	Robustesse occlusion	<code>np.random.rand</code>	ndarray modifie
	Mixing	Regularisation	<code>np.random.beta</code>	ndarray fusionne
	Time Stretching	Robustesse debit	<code>effects.time_s</code>	array float32
	Amplitude Scaling	Robustesse volume	<code>signal * gain</code>	array float32
	Noise Addition	Robustesse bruit	<code>np.random.norm</code>	array float32
	Reverberation	Robustesse acoustique	<code>convolve / decay</code>	array float32
	Time/Freq. Masking	Robustesse ASR	<code>melspectrogram</code>	spectrogramme

5.2 Erreurs Fréquentes et Solutions

Modalité	Erreur fréquente	Symptôme	Correction
Texte	Remplacement d'un mot-clé porteur du label	Label incohérent	Exclure une liste de mots protégés avant substitution

Modalité	Erreur fréquente	Symptôme	Correction
Texte	Ressources NLTK non téléchargées	LookupError	<code>nltk.download('wordnet')</code> avant appel
Image	Valeurs de pixels hors <code>[0,255]</code>	Image corrompue à l'affichage	Toujours <code>np.clip()</code> et <code>astype(np.uint8)</code>
Image	Rotation qui coupe les bords	Fond noir parasite	<code>borderMode=cv2.BORDER_REFLECT</code>
Image	<code>imread</code> retourne <code>None</code>	Crash sur <code>.shape</code>	Vérifier chemin, existence et droits fichier
Audio	Signal saturé (clipping)	Distorsion audible	<code>np.clip(y, -1.0, 1.0)</code> avant export
Audio	<code>sample rate</code> incohérent entre fichiers	Vitesse de lecture faussée	Forcer <code>sr</code> fixe : <code>librosa.load(sr=16000)</code>
Audio	Masquage excessif du spectrogramme	Perte d'information critique	Limiter <code>F</code> et <code>T</code> à moins de 20 % des dimensions

5.3 Bonnes Pratiques Générales

1. Choisir la technique selon la modalité

- Texte → Shuffling / Replacement / Insertion / Deletion
- Image → Géométrie / Couleur / Filtres / Erasing / Mixing
- Audio → Stretching / Bruit / Gain / Réverbération / Masking

2. Préserver le label

- Vérifier après transformation que le sens ou la classe reste inchangé
- Adapter l'intensité (probabilité, amplitude) au domaine applicatif

3. Ne jamais augmenter le jeu de test

- L'augmentation s'applique uniquement au jeu d'entraînement

4. Online vs Offline

- Offline : génération et stockage à l'avance (contrôle qualité facile)
- Online : application à chaque epoch d'entraînement (diversité maximale)

5. Doser l'intensité

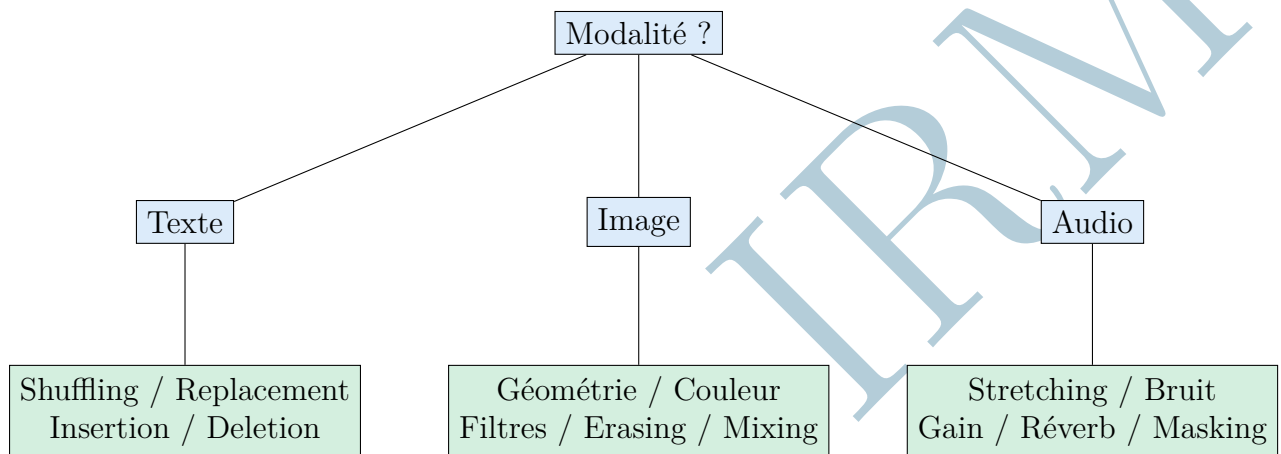
- Combiner 1 à 2 techniques maximum par échantillon
- Valider visuellement ou auditivement un échantillon avant intégration

5.4 Checklist de Reproductibilité

- ☐ Fixer la graine aléatoire : `random.seed(42)` / `np.random.seed(42)`.

- ☐ Sauvegarder tous les paramètres de transformation dans un fichier JSON.
- ☐ Versionner `requirements.txt` avec les versions exactes des bibliothèques.
- ☐ Tester chaque technique isolément avant intégration au pipeline complet.
- ☐ Vérifier qu'aucune valeur ne sort des bornes attendues (pixels, amplitude).
- ☐ Documenter la provenance de chaque donnée et les transformations appliquées.

5.5 Arbre de Décision



Conseil

Pour un projet multimodal, combinez les trois familles de techniques :

- Corpus texte enrichi : Shuffling + Replacement
- Dataset visuel diversifié : Transformations géométriques + Mixing
- Corpus sonore robuste : Noise Addition + Time/Frequency Masking

Chapitre 6 Bibliographie et Ressources

6.1 Ouvrages de Référence

Pour approfondir vos connaissances, voici les ouvrages et articles recommandés :

- Shorten, C., Khoshgoftaar, T.M. (2019). *A Survey on Image Data Augmentation for Deep Learning*. Journal of Big Data. — Référence complète sur les techniques d'augmentation d'images.
- Wei, J., Zou, K. (2019). *EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks*. EMNLP-IJCNLP. — Article fondateur des quatre techniques d'augmentation textuelle (shuffling, remplacement, insertion, deletion).
- DeVries, T., Taylor, G.W. (2017). *Improved Regularization of Convolutional Neural Networks with Cutout*. arXiv. — Article fondateur du random erasing.
- Zhang, H. et al. (2018). *mixup: Beyond Empirical Risk Minimization*. ICLR. — Article fondateur de la technique de mixing d'images.
- Park, D.S. et al. (2019). *SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition*. Interspeech. — Article fondateur du masquage temps/fréquence sur spectrogramme.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. — Chapitre sur la régularisation et l'augmentation de données en apprentissage profond.

6.2 Documentation Officielle

Outil	URL	Description
NLTK	nltk.org	Tokenisation, WordNet, ressources TAL
nlpaug	github.com/makcedward/nlpaug	Augmentation textuelle prête à l'emploi
OpenCV	docs.opencv.org	Vision par ordinateur, transformations image
Albumentations	albumentations.ai	Pipelines d'augmentation image optimisés
Librosa	librosa.org/doc	Analyse et transformation de signaux audio
Pydub	github.com/jiaaro/pydub	Manipulation audio haut niveau
audiomentations	github.com/iver56/audiomentations	Pipelines d'augmentation audio

6.3 Tutoriels Recommandés

- **Real Python** (realpython.com) — Tutoriels pratiques sur le traitement de texte, d'image et d'audio en Python.

- **Hugging Face** (huggingface.co/docs) — Documentation sur les techniques d’augmentation utilisées pour l’entraînement de modèles NLP et vision.
- **PyImageSearch** (pyimagesearch.com) — Guides détaillés sur l’augmentation d’images avec OpenCV et Keras.

6.4 Jeux de Données pour Pratiquer

- **IMDB Reviews** (ai.stanford.edu/~amaas/data/sentiment) — Corpus de critiques de films pour tester l’augmentation textuelle.
- **CIFAR-10** (cs.toronto.edu/~kriz/cifar.html) — 60 000 petites images en 10 classes, idéal pour tester les techniques d’augmentation d’images.
- **LibriSpeech** (www.openslr.org/12) — 960 heures d’anglais parlé, référence pour tester l’augmentation audio.
- **Common Voice** (commonvoice.mozilla.org) — Base audio multilingue, incluant le français.
- **UrbanSound8K** (urbansounddataset.weebly.com) — 8 732 extraits de sons urbains classés en 10 catégories.

6.5 Aspects Légaux et Éthiques

- **Biais introduits** — Une augmentation mal calibrée peut introduire des biais (ex. sur-représentation d’une orientation d’image) ; valider statistiquement le jeu augmenté.
- **Ne jamais augmenter le jeu de test** — Cela biaiserait l’évaluation et surestimerait la performance réelle du modèle.
- **Données sensibles** — Pour les visages ou la voix, vérifier que l’augmentation ne facilite pas une réidentification non consentie.
- **Licences** — Vérifier les licences des bibliothèques et des jeux de données utilisés avant toute diffusion commerciale.
- **Reproductibilité** — Toujours documenter les graines aléatoires et paramètres pour permettre la réplication des expériences.

Conseil

Pour un démarrage rapide :

- Texte : `nlpaug` (multi-techniques prêtes à l’emploi)
- Image : `albumentations` (rapide, composable, standard industriel)
- Audio : `audiomentations` (API simple, inspirée d’`albumentations`)